



TRUSTFLOW

Failure-Safe Digital Money Movement Platform

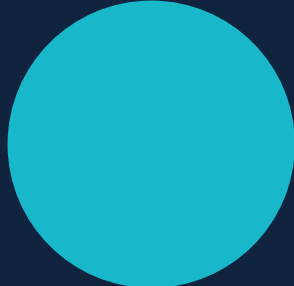
PSTU IT Carnival 2026 - Hackathon Challenge

Core promise: Money moves once. Users always know what happened.

A merged, build-ready plan covering the challenge scope, winning product concept, core features, reliability engineering, security, architecture, database, UI, demo, judge Q&A, and implementation priorities.

Scope:

Closed ecosystem • Simulated/fake funds • Small but thoughtful working product • No real-bank integration required



1. Executive Direction

The uploaded PSTU brief asks for a Money Movement Application that can transfer, request, and manage money in a closed ecosystem with fake balances. The key challenge is reliability: users and engineers must be able to trust the system when requests are duplicated, networks fail, multiple activities happen at once, or inputs are unexpected. The brief explicitly says to think beyond a simplistic CRUD app and to build a small but thoughtful working product. (Primary source: uploaded PSTU Hackathon Challenge brief, pp. 1-2.)

Winning product position

Do not pitch this as "another digital wallet". Pitch it as "a failure-safe money movement platform" where the difficult engineering problem is preventing incorrect money movement and clearly explaining transaction outcomes.

Layer	What the user sees	What judges should notice
Core wallet	Balance, Send, Request, History	Complete working money flow
Reliability	Processing / Recovered / Failed states	Real-world failure handling
Integrity	No duplicate transfer / no double spend	Idempotency + atomicity
Trust	Risk explanation + transaction timeline	Security and explainability
Demo mode	Chaos / reliability simulator	Team can prove the engineering

The project should feel small in scope but deep in thought.

- Baseline: Register/Login, fake balance, Send Money, Request Money, transaction history.
- Differentiators: idempotency, atomic balance protection, transaction state machine, transaction timeline, risk shield, recovery center, reliability simulator.
- Avoid scope traps: real bank APIs, payment gateways, blockchain, microservices, or a large collection of half-finished features.

North-star statement

"We are not optimizing only for successful transfers. We are optimizing for correct transfers under failure."

2. Product Concept - TRUSTFLOW

TrustFlow is a simulated digital wallet for a community of users. Each user starts with a fake balance. The product supports everyday money movement, but wraps every transaction in a reliability layer that protects against duplicates, concurrency, network uncertainty, and unexpected states.

Core user journeys

Journey	Flow	Expected outcome
Send money	Select user -> amount -> confirm -> transfer	One correct transfer + receipt
Request money	Choose user -> amount -> note -> request	Pending request visible to payer
Pay a request	Open request -> approve -> transfer	Request becomes paid
Group collection	Create amount -> invite members -> track	Paid / pending breakdown
Transaction issue	Open transaction -> Report Problem	Resolution / status guidance

Hero feature: Reliability Layer

Problem	TrustFlow behavior
User taps Send twice	Same idempotency key returns the same transaction result; no second debit.
Network timeout	Transaction can move to PROCESSING / UNKNOWN and later resolve through recovery.
Two spends happen concurrently	Database-level atomicity ensures the same balance cannot be spent twice.
Large/unusual payment	Risk Shield flags the transaction and can require extra confirmation.
User asks what happened	Transaction timeline shows each important event and final state.

Practical rule

AI should advise; deterministic backend rules should protect. The risk model may suggest that a transaction is unusual, but core money integrity must come from transactional database logic and explicit business rules.

3. Complete Feature Set - Prioritized

Build depth in the features below instead of adding many unrelated modules.

Priority	Feature	Why it matters	Demo value
P0	Auth + user accounts	Identity and ownership	High
P0	Fake balance	Required closed ecosystem	High
P0	Send Money	Primary challenge flow	High
P0	Request Money	Explicitly mentioned in brief	High
P0	Transaction History	Auditability and UX	High
P0	Idempotency	Prevents duplicate transfers	Very high
P0	Atomic transfer / ledger	Prevents corruption and double-spend	Very high
P0	Transaction state machine	Handles uncertainty	Very high
P1	Transaction timeline	Explains what happened	Very high
P1	Risk Shield	Trust + fraud-style controls	High
P1	Recovery / dispute center	Handles user-facing failure	Very high
P1	Reliability simulator	Lets judges see failure handling	Exceptional
P1	Group collection	Realistic everyday use case	Medium-high
P2	QR / contacts / notifications	Polish only	Medium

UI-level details that should be visible

- Send: recipient, amount, note, confirmation, risk status, protected-transfer message.
- Request: payer, amount, note, expiry/status, accept/reject.
- Transaction detail: amount, sender, receiver, status, transaction ID, timestamps, timeline, proof/fingerprint, report-problem action.
- Dashboard: available balance, send/request actions, recent activity, pending requests, and system protection status.

4. Reliability Engineering - The Winning Differentiator

The brief specifically warns that users can act quickly, networks can behave unexpectedly, multiple activities can happen simultaneously, and usage can go beyond initial assumptions. This is the strongest signal in the challenge. (Uploaded brief, p. 2.)

A. Transaction State Machine

CREATED -> PROCESSING -> SUCCESS

CREATED -> PROCESSING -> FAILED

CREATED -> PROCESSING -> UNKNOWN -> RECOVERING -> SUCCESS / FAILED

Never show the user a misleading instant "failed" result when the backend does not actually know whether the transfer committed.

B. Idempotency

Every money-changing request gets an idempotency key. If the same key is received again, the backend returns the original transaction result instead of performing a second transfer. Store the key with a UNIQUE constraint.

Example: User taps Send three times -> API receives three identical requests -> database records one transaction -> subsequent requests return that transaction.

C. Atomic transfer

The debit and credit must succeed together or not at all. Use a database transaction with a protected balance check. The system must prevent negative balance caused by race conditions.

D. Concurrency test

Starting balance = 5,000. Two simultaneous requests each attempt to send 4,000. Correct result: one succeeds and one fails with insufficient balance. Both must never succeed.

E. Audit trail

Store transaction events such as CREATED, VALIDATED, PROCESSING, DEBITED, CREDITED, COMPLETED, FAILED, and RECOVERY_STARTED. This powers both debugging and the user-facing timeline.

5. Ledger, Security, Risk & Recovery

Ledger model

Do not make a balance overwrite the only record of truth. Keep auditable transaction/ledger history so every movement can be explained and reconstructed.

Table	Key fields	Purpose
users	id, name, phone/email, password_hash	Identity
accounts	id, user_id, balance, status	Wallet account
transactions	id, sender_id, receiver_id, amount, status, idempotency_key, risk_score	Transfer record
ledger_entries	id, transaction_id, account_id, entry_type, amount	DEBIT/CREDIT audit
transaction_events	id, transaction_id, event_type, metadata, created_at	Timeline / audit
money_requests	id, requester_id, payer_id, amount, status, note, expires_at	Request flow
risk_events	id, transaction_id, reason, score	Explainable risk signals

Security minimum

- Hash passwords and transaction PINs; never store plaintext secrets.
- Authenticate every sensitive request and authorize access to the actual object being requested.
- Validate amount, recipient, transaction state, and request ownership on the server.
- Rate-limit login, PIN attempts, and high-frequency endpoints.
- Use unique transaction IDs and idempotency keys; never trust a client-provided balance.

Risk Shield - simple, explainable rules

Signal	Example score
New recipient	+30
Unusually large amount	+25
Many transfers in short time	+20
Repeated failed PIN attempts	+15
Unusual transfer frequency	+10

Risk bands: 0-30 Low, 31-60 Medium, 61-100 High. Show reasons, not just a number.

Recovery Center

Handle "money deducted but recipient did not receive", "transaction stuck", "unrecognized transaction", and "duplicate attempt". Tie every case to the transaction ID and its ledger/timeline state.

6. Recommended Architecture

Use a modular monolith for the hackathon. It is easier to build, test, explain, and deploy than microservices while still keeping clear domain boundaries. A later scale-out path can split services when needed.

React Frontend

Dashboard | Send | Request | History | Transaction Detail | Reliability Center

↓

Node.js + Express API

Auth Module | Transfer Engine | Request Engine | Risk Engine | Recovery Module

↓

PostgreSQL

Users | Accounts | Transactions | Ledger Entries | Transaction Events | Requests | Risk Events

Recommended stack

Layer	Choice	Reason
Frontend	React + Tailwind CSS	Fast UI iteration and polished hackathon screens
Backend	Node.js + Express	Simple API development and clear business logic
Database	PostgreSQL	Transactions, constraints, row locking, strong consistency model
Auth	JWT or secure session	Straightforward prototype authentication
Testing	API tests + scripted failure cases	Prove money integrity, not only happy paths

Core transfer API idea

POST /transactions

Header: Idempotency-Key: UUID

Body: { recipient_id, amount, note }

GET /transactions/:id

Returns current status + timeline

POST /money-requests

Creates a request

POST /money-requests/:id/pay

Accepts and executes the request

Scale story for judges

The prototype can stay a modular monolith. A future 10M+ user environment can add horizontal API instances, cache read-heavy views, queue notifications/asynchronous work, and separate write-intensive transaction processing from other workloads.

7. UI / UX Blueprint

The product should feel trustworthy at every money-changing step.

Screen	Important UI elements
Login	Logo, phone/email, password, login, concise security reassurance
Dashboard	Available balance, Send, Request, recent activity, pending requests, protection status
Send Money	Recipient search, amount, note, Continue
Confirm Transfer	Recipient identity, amount, risk level, protection indicators, Send Securely
Transaction Detail	Status, amount, transaction ID, timeline, verification/proof, Report Problem
Request Money	Payer, amount, note, due/expiry, request status
Recovery Center	Issue categories, current state, next step, resolution evidence
Reliability Simulator	Duplicate request, network delay, timeout, concurrent transfer, invalid input toggles

Design language

- Use clear status chips: PROCESSING, SUCCESS, FAILED, RECOVERING, UNKNOWN.
- Show transaction IDs and timestamps prominently enough to support support/debugging.
- Avoid vague errors such as "Something went wrong" when the system knows the actual state.
- Keep the dashboard simple; put engineering depth inside Transaction Detail and Reliability Center.

Sample transaction detail

COMPLETED

2,500

Rahim -> Tasir

Transaction: TXN-A81F92

09:42:01 Created

09:42:02 Verified

09:42:03 Ledger updated

09:42:04 Completed

[View Proof] [Report Problem]

8. Winning Demo Script

Do not spend the demo on registration. Get quickly to the engineering moments that answer the brief.

Scene	What you do	What judges should see
1. Normal send	Send 2,500 from User A to User B	Success + receipt + timeline
2. Money request	Request 1,200 and pay it	Request lifecycle
3. Duplicate request	Enable simulator -> Send same request 3 times	Exactly one transfer
4. Network timeout	Enable timeout -> submit transfer	PROCESSING/UNKNOWN -> recovery -> final state
5. Concurrent spend	Two tabs attempt 4,000 from a 5,000 balance	One success, one insufficient-balance failure
6. Risky transaction	Large/new-recipient transfer	Explainable risk alert + additional confirmation

Single strongest line

"The hard part of money movement is not moving the money. It is making sure retries, concurrency and failures do not move it incorrectly - and making the final state explainable to the user."

90-second technical explanation

- Every transfer receives an idempotency key.
- The sender balance is checked and updated inside one database transaction.
- Debit and credit are committed atomically.
- Every important state transition is logged.
- Failures can resolve through the recovery path instead of producing a misleading final message.
- Risk rules add a trust layer without becoming the source of truth for money.

9. Build Plan for the Hackathon

Because the brief emphasizes a small but thoughtful working product, treat time as a hard constraint. Build the money path first; polish only after reliability is real.

Stage	Build	Definition of done
Stage 1	Auth + accounts + seed balances	Two users can log in and see balances
Stage 2	Send + request + history	Normal flows fully work
Stage 3	Transactions + idempotency + atomicity	Duplicate and race-condition tests pass
Stage 4	Timeline + recovery + risk shield	Failure states are visible and explainable
Stage 5	Reliability simulator + UI polish	Judges can trigger edge cases live
Final	Demo + pitch + Q&A; prep	No broken path in the scripted demo

Team split - 4 people

Role	Primary ownership
Frontend	Dashboard, send/request, transaction detail, responsive UI
Backend	Auth, users, transactions, requests, validation
Reliability	Ledger, idempotency, concurrency, transaction states, recovery
Risk / Demo	Risk engine, simulator, test scripts, presentation, UI polish

Definition of done

- A transfer cannot debit the sender twice for the same request.
- A concurrent double-spend attempt cannot push the account below the allowed balance.
- Every completed transfer has a unique transaction ID and an auditable event trail.
- The user can see whether a transaction is successful, failed, processing, unknown, or recovering.
- Request Money works end-to-end.
- The demo can intentionally create duplicate, timeout, and concurrency scenarios.

10. Judge Q&A; Cheat Sheet

Likely question	Best answer
Why not a normal wallet?	"The challenge is explicitly about correctness under unreliable conditions. Our core value is failure-safe money movement, not just CRUD."
Why PostgreSQL?	"We need transactional consistency, constraints and safe concurrent balance updates. It fits the money-movement core naturally."
Why modular monolith?	"It keeps the hackathon system simple to operate while preserving domain boundaries and a future scale-out path."
What if the user taps Send twice?	"The same idempotency key maps both requests to one transaction result, so money moves once."
What if two withdrawals happen simultaneously?	"The balance check and update happen atomically, so only valid spending can commit."
Why use AI?	"AI is a risk signal, not the source of truth. Deterministic backend logic protects the ledger."
How can this scale?	"Stateless APIs, indexed storage, caching for reads, asynchronous notifications, and eventual decomposition of hot transaction paths."
Why no blockchain?	"The challenge does not require it. Strong transactional guarantees and auditability are more direct for this prototype."

One-slide pitch

TRUSTFLOW
Failure-Safe Digital Money Movement

Send. Request. Recover. Verify.

Protect every transaction from duplicate requests, concurrent spending and uncertain network states - while showing users exactly what happened.

Winning position

Useful enough for users. Explainable enough for judges. Defensible enough for engineers.

11. Final Blueprint + Reference Notes

Recommended MVP feature bundle

Auth -> Dashboard -> Send Money -> Request Money -> Transaction History -> Transaction Detail -> Idempotency -> Atomic Ledger -> Transaction State Machine -> Risk Shield -> Recovery Center -> Reliability Simulator

What the uploaded brief requires

- Money Movement Application that can move / transfer / request money between users.
- A realistic closed ecosystem with fake balances and users who rely on correctness and trust.
- Think beyond simplistic CRUD and account for unexpected input, unreliable networks, multiple simultaneous activities, and usage beyond initial expectations.
- No restriction on technology; AI-assisted development is permitted, but the team must understand and defend its engineering decisions.
- No need for full banking integration; focus on a small but thoughtful working product.
- Users may be given an initial BDT 100,000 fake balance upon registration.

External engineering references used for the design rationale

Stripe API - Idempotent requests: https://docs.stripe.com/api/idempotent_requests

OWASP API Security Top 10 (2023): <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

AWS Prescriptive Guidance - Retry / idempotency concepts:

<https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/retry-backoff.html>

AWS Prescriptive Guidance - Transactional outbox pattern:

<https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html>

Bangladesh Bank ICT Security Guideline for Financial Institutions (2023):

https://www.bb.org.bd/aboutus/regulationguideline/brpd/jun192023_ictsecurityv4.pdf

bKash products / Request Money reference: <https://www.bkash.com/en/products-services>

Source note

The PSTU Hackathon Challenge brief is the primary source for scope and problem framing in this document. The product and engineering recommendations are the merged solution direction derived from that brief plus general software/payment-system design practices; they are recommendations, not additional requirements imposed by PSTU.

Final recommendation

Build TrustFlow, and make reliability the product. A polished happy-path wallet is easy to copy. A wallet that proves it remains correct when users retry, networks fail, and two actions collide is much closer to the engineering problem the challenge is asking you to solve.